



SERVIÇO PÚBLICO FEDERAL · MINISTÉRIO DA EDUCAÇÃO  
UNIVERSIDADE FEDERAL DE VIÇOSA · UFV  
CAMPUS FLORESTAL

# Trabalho Prático - Parte 2: Caldeirão

**Autores (as):**

Sabrina Bruni de Souza Faria [EF 05081]

Luiz Guilherme de Sá Gontijo [EF 04839]

Breno Júnio de Oliveira Gomes [EF 05085]

**Disciplina:** Sistemas Distribuídos e Paralelos [CCF - 355]

# Sumário

|                                                           |          |
|-----------------------------------------------------------|----------|
| <b>1. Introdução:</b>                                     | <b>3</b> |
| <b>2. Modelo Físico:</b>                                  | <b>3</b> |
| <b>3. Modelo de Arquitetura:</b>                          | <b>3</b> |
| 3.1. Paradigma de Comunicação:                            | 3        |
| 3.2. Modelo de Arquitetura Básico:                        | 4        |
| 3.3. Funções e responsabilidades das entidades do modelo: | 4        |
| 3.4. Camadas de arquitetura:                              | 6        |
| <b>4. Modelos Fundamentais:</b>                           | <b>7</b> |
| 4.1. Modelo de Interação:                                 | 7        |
| 4.2. Modelo de Falhas:                                    | 7        |
| 4.3. Modelo de Segurança:                                 | 7        |
| <b>5. Referências:</b>                                    | <b>8</b> |

# 1. Introdução:

O objetivo deste documento é detalhar a modelagem do Caldeirão, documentando a modelagem física, a escolha do paradigma de comunicação, do modelo de arquitetura básico, especificando as funções e responsabilidades do sistema, a estruturação física e lógica do sistema, e determinando as modelagens de interação, falhas e segurança. Como o projeto ainda está em fase preliminar, pode ser que uma ou outra característica aqui descrita passe por alterações, mas o objetivo é cumprir com estas definições.

## 2. Modelo Físico:

A modelagem física escolhida para nosso Marketplace será de escala de internet, pois diversos usuários poderão usá-la, de diversos lugares, o que só é possível através da Internet e protocolos de comunicação mais abstraídos. O modelo primitivo não nos atende, pois nele, é esperado que os usuários sejam conhecidos, pertencentes a uma rede fechada, e o de ultra-larga escala não faz sentido, pois requer garantias de qualidade, disponibilidade e redundância muito além da demanda que esperamos para o sistema.

## 3. Modelo de Arquitetura:

### 3.1. Paradigma de Comunicação:

O paradigma **Fila de Mensagens** é um modelo assíncrono e de baixo acoplamento. Em um marketplace, como é o caso do nosso sistema, operações como finalização de pedidos, atualização da disponibilidade, ou processamento de pagamentos podem funcionar via filas, evitando bloqueios da interface do usuário. Esse paradigma é resiliente e as mensagens persistem até serem processadas mesmo que o servidor falhe temporariamente, fornecendo uma camada de confiabilidade extra ao sistema. Outra vantagem – mais importante para sistemas de marketplace em geral – é que as filas permitem gerenciar picos de demanda sem sobrecarregar o servidor. Entendemos que esse paradigma é o mais adequado para nosso sistema.

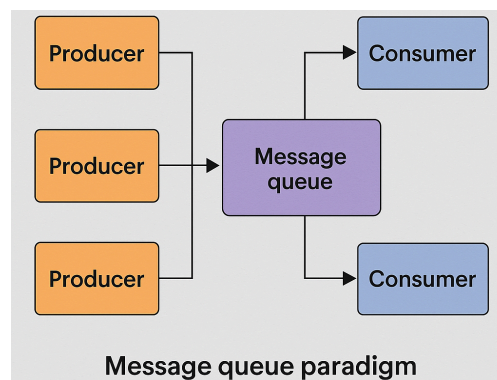


Figura 1: Paradigma de Filas de Mensagem

### 3.2. Modelo de Arquitetura Básico:

As principais funcionalidades do nosso sistema consistem em consultas: Um usuário quer informações de um produto ou loja, ou informações sobre si mesmo. Ou então, o usuário quer que outros possam ter informações sobre seu produto (criar um anúncio). Esses casos se encaixam melhor no modelo básico de **cliente-servidor**, pois nele, inúmeros clientes poderão obter ou fornecer as informações que estiverem disponíveis ao servidor, e este fornecerá respostas atualizadas com as informações que eles solicitaram. A única outra funcionalidade importante que não se encaixa tão trivialmente nesse modelo é a efetuação de uma compra. Mas ainda sim, quando um usuário deseja comprar um produto, o processo cliente pode apenas solicitar a compra ao servidor, e manter uma troca de mensagens até a finalização da compra, mantendo o usuário sem conhecimento do sistema. Após o pagamento, o servidor começa a tratar a confirmação do pagamento com o processo cliente do vendedor. Sendo assim, de fato, essa é a arquitetura básica que melhor se enquadra com as necessidades do nosso sistema.

### 3.3. Funções e responsabilidades das entidades do modelo:

As entidades **Cliente** e **Servidor** estarão associadas a todos os casos de uso, ou seja, terão as mesmas funcionalidades, mas cada uma de uma forma diferente, isto significa que terão responsabilidades diferentes para cada caso de uso.

No caso de uso **Cadastrar-se**, o **Cliente** será responsável por solicitar esse cadastramento, receber os campos necessários, enviar as informações solicitadas, receber uma solicitação de código de confirmação, enviar o código e receber uma resposta, enquanto o **Servidor** será responsável por receber a solicitação de cadastramento, enviar os campos necessários, receber os campos preenchidos, conferir as informações com o banco de dados, enviar um código de confirmação via e-mail, enviar uma solicitação de código para o **Cliente**, receber o código, conferir o código, enviar uma resposta e realizar o cadastramento adicionando o usuário no banco de dados.

No caso de uso **Realizar login**, o **Cliente** será responsável por solicitar esse login ou solicitar acesso a uma funcionalidade que exija identificação, receber os campos necessários, enviar as informações solicitadas e receber uma resposta, enquanto o **Servidor** será responsável por receber essa solicitação, enviar os campos necessários, receber os campos preenchidos, conferir as informações com o banco de dados e enviar uma resposta.

No caso de uso **Visualizar Produto**, o **Cliente** será responsável por solicitar os detalhes de informação de um produto e receber uma resposta com essas informações, enquanto o **Servidor** será responsável por receber essa solicitação, procurar o produto no banco de dados e enviar uma resposta com as informações do produto. As responsabilidades são as mesmas para o caso de uso **Visualizar**

**Loja**, porém com a solicitação e resposta das informações da loja, ao invés do produto.

No caso de uso **Pesquisar**, o **Cliente** será responsável por solicitar uma pesquisa por um texto e receber uma resposta com os produtos e lojas que se encaixam nesse texto, enquanto o **Servidor** será responsável por receber essa solicitação, procurar por produtos e lojas com esse texto no banco de dados e enviar uma resposta dos produtos e lojas que se encaixam na pesquisa.

No caso de uso **Criar Loja**, o **Cliente** será responsável por solicitar a criação de uma loja, receber os campos necessários, enviar as informações solicitadas e receber uma resposta, enquanto o **Servidor** será responsável por receber essa solicitação, enviar os campos necessários, receber os campos preenchidos, conferir as informações, adicionar no banco de dados e enviar uma resposta. O caso de uso **Criar Produto** é bem parecido, a diferença é que o **Cliente** realiza a solicitação da criação de um produto também enviando qual a loja referente e o **Servidor** não tem informações a conferir. Além disso, o caso de uso **Criar Anúncio** também é bem parecido, a diferença é que o **Cliente** realiza a solicitação da criação de um anúncio também enviando qual o produto referente.

No caso de uso **Editar Loja**, o **Cliente** será responsável por solicitar a edição de uma loja, receber os campos necessários com as informações atuais da loja, enviar as novas informações e receber uma resposta, enquanto o **Servidor** será responsável por receber essa solicitação, enviar os campos preenchidos, receber os campos editados, conferir as informações, editá-las no banco de dados e enviar uma resposta. O caso de uso **Editar Perfil** é exatamente igual, com a diferença que a solicitação de edição é para perfil e não para loja. O caso de uso **Editar Produto** é bem parecido, sendo a diferença que o **Cliente** realiza a solicitação da edição de um produto também enviando qual a loja referente e o **Servidor** não tem informações a conferir. O caso de uso **Editar Anúncio** também é bem parecido, porém nele o **Cliente** realiza a solicitação da edição de um anúncio também enviando qual o produto referente.

No caso de uso **Comprar Produto**, o **Cliente** será responsável por solicitar a compra de um produto, receber os campos necessários, enviar as informações solicitadas, receber as informações de pagamento, confirmar pagamento e receber uma resposta, enquanto o **Servidor** será responsável por receber essa solicitação, conferir se o usuário está identificado, enviar os campos necessários, receber os campos preenchidos, enviar as informações de pagamento, receber confirmação de pagamento, alterar o banco de dados, enviar uma resposta e enviar um email para o vendedor. Se o usuário não estiver identificado, o **Servidor** será responsável por enviar uma solicitação de login e seguir com as responsabilidades do caso de uso **Realizar login**.

No caso de uso **Vender Produto**, o **Cliente** será responsável por solicitar a confirmação de pagamento de um pedido e receber uma resposta, enquanto o **Servidor** será responsável por receber essa solicitação, enviar um email ao comprador, alterar o banco de dados e enviar uma resposta.

### 3.4. Camadas de arquitetura:

A fim de apresentar a modelagem da arquitetura em camadas física e lógica de maneira mais sucinta e didática, decidimos desenhar esquematizações das camadas. A estrutura de camadas físicas se encontra na figura Figura 2, e a estrutura de camadas lógicas, na Figura 3.

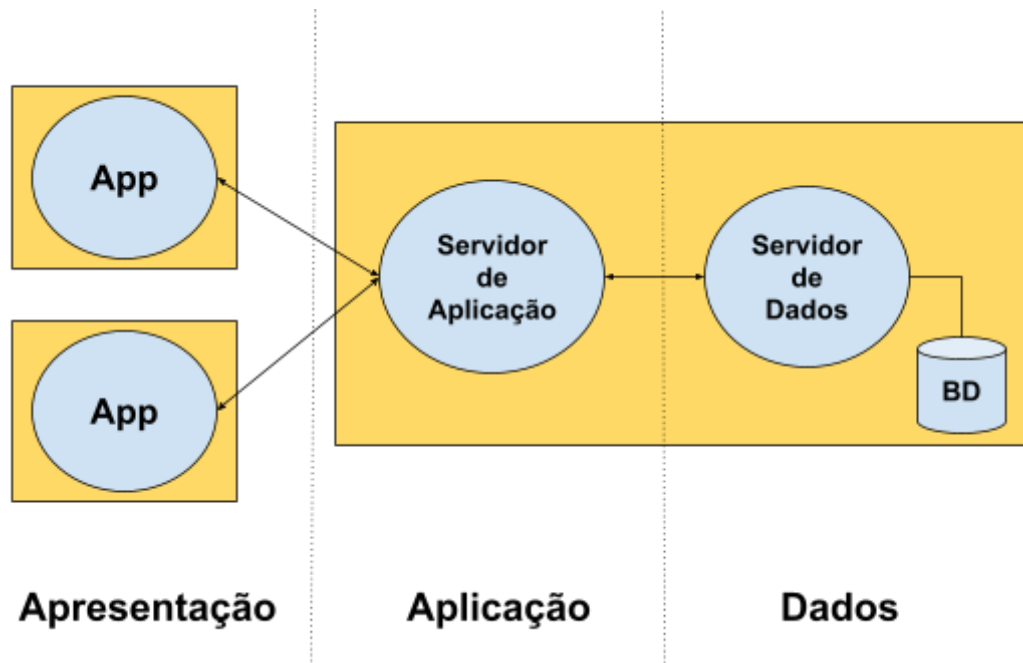


Figura 2: Camadas Físicas

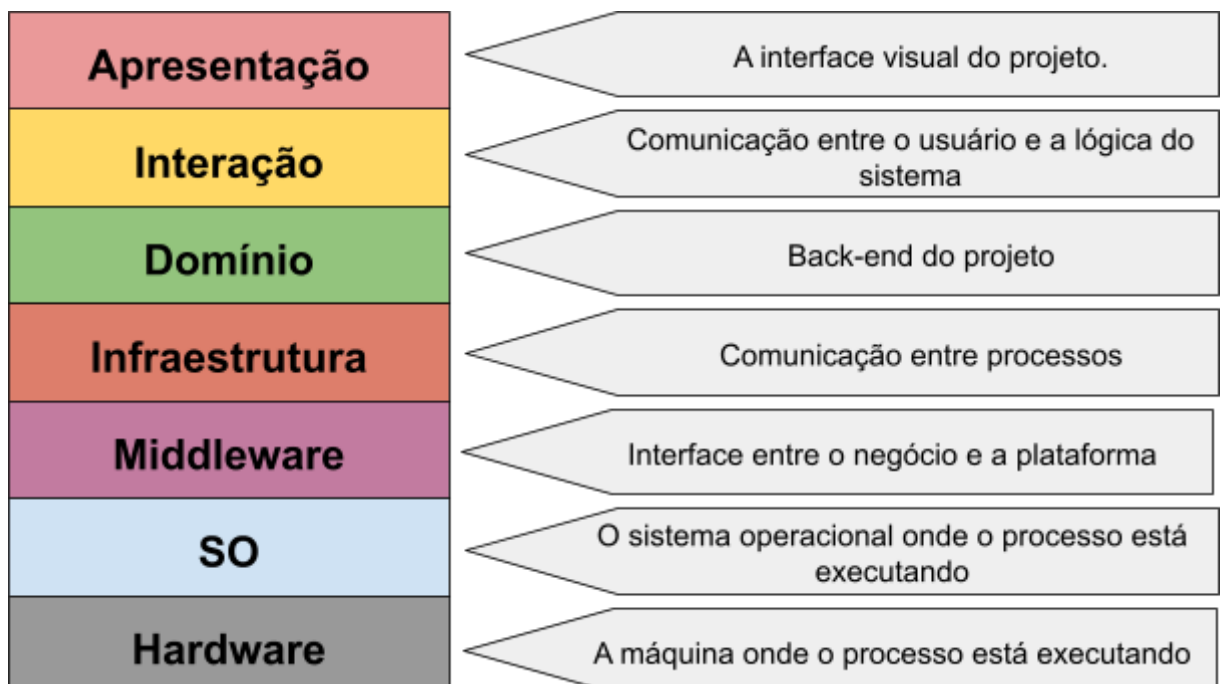


Figura 3: Camadas Lógicas

## 4. Modelos Fundamentais:

### 4.1. Modelo de Interação:

Nosso sistema utilizará modelo **assíncrono** de comunicação, uma vez que as trocas de mensagens não precisam ser instantâneas. É um sistema de Marketplace, não será problemático se o carregamento da resposta de uma busca demorar 2000 ms, por exemplo.

No funcionamento do Caldeirão, a rede transportará não apenas diversos números e strings relativamente longas, mas também imagens. Para isso, é ideal que a **taxa de transmissão de dados** seja a mais alta e a **latência** e o **jitter** sejam os mais baixos o possível, mas nenhuma dessas métricas é essencial para o funcionamento correto do nosso serviço. O mais importante dos três é a taxa de **transmissão de dados**, pois uma taxa muito baixa pode acarretar num carregamento muito lento das informações trazidas pelo servidor, e isso pode ser frustrante para os usuários, embora não impossibilite o uso do sistema. Já a ocorrência de **jitter** e **latência** alta são problemas maiores em aplicações síncronas, o que não é o caso do nosso sistema. Se ocorrer de por alguns segundos ficar mais difícil carregar determinada informação, não fará tanta diferença para o usuário, pois são apenas informações e imagens.

### 4.2. Modelo de Falhas:

**Falhas por omissão** são falhas em que um processo ou canal de comunicação deixa de funcionar como deveria. Quando ocorre uma falha por omissão na comunicação, por exemplo, uma mensagem pode se perder ao haver uma falha na rede no momento em que uma requisição estiver sendo feita para um processo servidor. Para o problema deste exemplo, o paradigma Fila de Mensagens já ajuda a tratá-lo na natureza de sua implementação: as mensagens são levadas por um processo cliente a uma fila e posteriormente um processo servidor recebe essa mensagem. Se houver um problema de rede no momento da requisição – por exemplo, a rede ficar fora do ar quando um usuário for comprar uma poção –, essa requisição fica armazenada na fila até que o servidor possa recebê-la. O servidor (ou consumidor da mensagem) deve enviar uma confirmação explícita de recebimento da mensagem, e, caso isso não aconteça, a mensagem pode ser reprocessada após um tempo limite.

### 4.3. Modelo de Segurança:

O sistema necessita de mecanismos de **proteção aos processos**. Como o sistema de marketplace lida com transações, e o nosso sistema terá autenticação de usuários, é essencial proteger: o processo cliente de invasores fazendo *spoofing* e se passando pelo processo servidor para roubar as credenciais dos usuários; o processo servidor de responder requisitos de um invasor.

Assim como o nosso marketplace precisa de proteção nos processos para evitar “acessos indevidos” com invasores burlando o controle de acesso, ele também precisa **proteger as mensagens trocadas** pelos processos. Um sistema de vendas exige segurança nas informações das transferências. Assim, os canais de comunicação não podem estar sujeitos à alteração ou injeção de mensagens indevidas.

Pesquisas nos retornaram o protocolo criptográfico Transport Layer Security (TLS) como uma opção para lidar com ambos os casos. O TLS serve como uma camada de segurança entre o socket TCP e o nosso sistema. Com uma chave secreta entre o cliente e o servidor, as mensagens ficam criptografadas (lidando com a proteção do canal de comunicação). Quando cliente e servidor estabelecerem uma conexão com TLS, o servidor apresenta um certificado digital (oferecendo uma camada de proteção contra invasores tentando se passar pelo servidor).

## 5. Referências:

[1] **COULOURIS**, George. Sistemas Distribuídos: Conceitos e Projeto. 5. ed. São Paulo: Pearson Education do Brasil, 2013.

[2] **OPENAI**. *ChatGPT*. [S. l.]: OpenAI, 2025. Disponível em: <https://chat.openai.com/>.